

Project Interim Report
On
Bank Account Management System
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming With Java

SESSION: 2025-26



Submitted By
Harsh Kumar
10550

**Bachelor of Technology in Computer Science &
Engineering with AI and ML in association with Google**

Under the Guidance of
Mr. Soumik Karmakar

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Acknowledgment:

I would like to express my sincere gratitude to my respected teacher, Mr. Soumik Karmakar, for his valuable guidance, continuous support, and encouragement throughout the development of this project. His insightful suggestions and clear explanations helped me understand the core concepts of Object-Oriented Programming and apply them effectively in this project. Without his guidance, completing this work successfully would not have been possible.

I am also thankful to Rungta International Skills University for providing me with the opportunity to work on this project as a part of my academic curriculum. The university has offered a supportive learning environment, quality resources, and a platform to enhance my technical knowledge and practical skills.

Additionally, I would like to acknowledge all the faculty members and staff who have directly or indirectly contributed to my learning and development during this project. Their support and motivation have played an important role in the successful completion of this work.

Lastly, I would like to thank everyone who supported me during this project and encouraged me to give my best effort. This project has been a great learning experience, and I am grateful for all the guidance and opportunities provided to me.

Abstract:

In the era of digital transformation, banking systems have evolved significantly to ensure secure, efficient, and reliable management of financial transactions. One of the fundamental components of such systems is the effective handling of bank accounts, which requires strict data protection, accurate processing, and robust validation mechanisms. This project, titled “Design and Implementation of a Bank Account Management System using Object-Oriented Programming (OOP)”, aims to develop a simplified yet functionally sound banking model that demonstrates the practical application of core OOP principles.

The primary objective of this project is to design a secure and user-friendly system that manages bank account operations such as deposits and withdrawals while ensuring data integrity. The system is developed using Object-Oriented Programming concepts, with a strong focus on encapsulation, data hiding, and controlled access to sensitive information.

The account balance is maintained as a private data member, preventing unauthorized access and modification, thereby enhancing the security of financial data. The system provides well-defined public methods to perform operations such as depositing and withdrawing money. Each transaction is validated to ensure correctness and prevent invalid operations. For instance, the system restricts negative deposits and prevents withdrawals that exceed the available balance, thereby avoiding overdraft situations. These validation checks not only improve the reliability of the system but also simulate real-world banking constraints.

Additionally, the project demonstrates the importance of method design and object interaction in building scalable and maintainable software. By creating instances of the BankAccount class and performing various operations, users can observe how objects interact within a controlled environment. The use of constructors further enhances the flexibility of the system by allowing initial balance setup during object creation.

This project serves as a foundational learning model for students to understand how theoretical programming concepts are applied in real-world scenarios. It emphasizes the significance of secure coding practices, input validation, and modular design. Moreover, it helps in developing problem-solving skills and logical thinking required for software development.

In conclusion, the Bank Account Management System successfully illustrates the implementation of essential OOP concepts in a practical context. It provides a clear understanding of how secure and efficient systems can be built by integrating proper design principles, making it a valuable educational tool for aspiring programmers.

Introduction:

In today's rapidly advancing digital world, the banking sector plays a crucial role in managing financial transactions efficiently and securely. With the increasing reliance on computerized systems, it has become essential to design applications that not only perform operations accurately but also ensure the safety and integrity of sensitive financial data. This project, "Bank Account Management System using Object-Oriented Programming (OOP)", is developed to simulate basic banking functionalities while emphasizing secure and structured programming practices.

A bank account management system is a fundamental component of modern banking software that allows users to manage their accounts by performing operations such as deposits, withdrawals, and balance inquiries. In real-world applications, such systems are designed with high levels of security and reliability to prevent unauthorized access and ensure correct transaction processing. This project replicates these essential features in a simplified form, making it easier for students to understand the underlying concepts and logic.

The primary aim of this project is to implement a system that demonstrates the use of Object-Oriented Programming principles, particularly encapsulation and data hiding. Encapsulation is one of the key features of OOP, which involves bundling data and methods that operate on that data within a single unit, known as a class. In this project, the BankAccount class is used to encapsulate the account balance and related operations, ensuring that the balance cannot be accessed or modified directly from outside the class. This approach enhances data security and maintains the integrity of the system. Another important aspect of the project is the implementation of controlled methods for performing transactions. The system provides dedicated functions for depositing and withdrawing money, each designed with proper validation checks. These validations ensure that only valid transactions are processed, such as preventing negative deposits and restricting withdrawals that exceed the available balance. Such checks are essential in real banking systems to avoid errors and fraudulent activities.

Furthermore, this project introduces students to the concept of object interaction, where multiple objects can be created and used to simulate real-life scenarios. By interacting with the system through method calls, users can perform various operations and observe how the system responds to different inputs. This helps in building a strong understanding of how software components communicate and function together.

The project also highlights the importance of writing clean, modular, and maintainable code. By organizing the program into classes and methods, the system becomes easier to understand, debug, and extend. These practices are essential for developing large-scale applications in the software industry.

In conclusion, the Bank Account Management System serves as an introductory yet comprehensive project for understanding the practical implementation of Object-Oriented Programming concepts. It bridges the gap between theoretical knowledge and real-world application, providing students with valuable insights into secure system design, logical problem-solving, and efficient coding techniques. This project lays a strong foundation for developing more complex and advanced software systems in the future.

Objectives:

- To design and develop a Bank Account Management System using Object-Oriented Programming concepts.
- To understand and implement the concept of encapsulation by securing the account balance using private data members.
- To develop methods for performing basic banking operations such as deposit, withdrawal, and balance inquiry.
- To ensure proper validation of user inputs to prevent invalid transactions like negative deposits and overdrafts.
- To demonstrate the use of classes and objects in solving real-world problems.
- To enhance understanding of method design and controlled access to data through public functions.
- To simulate real-life banking scenarios through object interaction and transaction handling.
- To improve logical thinking and problem-solving skills in programming.
- To write clean, structured, and modular code for better readability and maintainability.
- To gain practical knowledge of how OOP concepts are applied in real-world software development.

Scope:

- The project focuses on designing and implementing a Bank Account Management System using Object-Oriented Programming principles in a structured and systematic manner.
- It includes the creation of a BankAccount class that represents a real-world bank account and handles all related operations.
- The system ensures secure handling of financial data by using encapsulation, where the account balance is kept private and cannot be accessed directly from outside the class.
- It provides basic banking functionalities such as deposit, withdrawal, and balance inquiry, which form the core operations of the system.
- The project emphasizes proper validation of transactions to prevent invalid operations such as negative deposits and withdrawals exceeding the available balance.
- It demonstrates how controlled access to data can be achieved through public methods, ensuring that all modifications to the balance are done securely.
- The system allows users to simulate real-life banking transactions by creating objects and performing multiple operations on them.
- It incorporates the use of constructors for initializing account balance, making the system more flexible and realistic.
- The project highlights the importance of modular programming by dividing tasks into separate methods, improving readability and maintainability of the code.
- It includes handling of different scenarios and edge cases, ensuring that the system behaves correctly under various conditions.
- The implementation is limited to a console-based application, keeping the design simple and focused on core programming concepts.
- The project does not include advanced features such as database connectivity, graphical user interface, or networking, but it provides a base for future enhancements.
- It serves as an educational tool to help students understand how Object-Oriented Programming concepts can be applied in real-world applications.
- Overall, the scope of this project is to provide a secure, simple, and practical implementation of basic banking operations while strengthening the understanding of OOP concepts.

System Requirement:

The Bank Account Management System is a simple console-based application developed using Java and does not require high-end hardware or advanced software configurations.

The system requirements are categorized into hardware and software requirements as follows:

1. Hardware Requirements

- Processor: Minimum Intel Pentium IV or above
- RAM: At least 2 GB (4 GB recommended for smooth performance)
- Hard Disk: Minimum 100 MB of free space
- Input Devices: Keyboard and Mouse
- Output Device: Monitor

These basic hardware specifications are sufficient to run the program efficiently, as the application is lightweight and does not involve heavy processing or graphical operations.

2. Software Requirements

- Operating System: Windows 7 or above / Linux / macOS
- Programming Language: Java
- Development Kit: JDK (Java Development Kit) 8 or above
- IDE/Editor: Eclipse / NetBeans / IntelliJ IDEA / VS Code (optional)
- Compiler: Java Compiler (included in JDK)

The system is developed using Java, which makes it platform-independent. As long as the Java Runtime Environment (JRE) is installed, the program can be executed on any operating system.

3. Functional Requirements

- The system must allow users to create a bank account with an initial balance.
- It should support deposit and withdrawal operations.
- The system must validate user inputs to prevent invalid transactions.
- It should display the current account balance when requested.
- It must ensure that the balance cannot be accessed or modified directly from outside the class.

4. Non-Functional Requirements

- The system should be simple and easy to use.
- It must ensure data security using encapsulation.
- The program should execute efficiently without delays.
- The code should be readable, modular, and maintainable.

Overall, the system requirements for this project are minimal, making it easy to develop, run, and test on any standard computer system while focusing on learning core programming concepts.

Methodology:

The development of the Bank Account Management System follows a systematic approach based on Object-Oriented Programming principles to ensure clarity, security, and efficiency in handling banking operations. The methodology adopted in this project involves a step-by-step process, starting from system design to implementation and testing.

Initially, the problem is analyzed to identify the core requirements of the system. The primary functionalities include managing account balance, performing deposit and withdrawal operations, and ensuring secure handling of financial data. Based on these requirements, a suitable design is prepared using OOP concepts, where the system is modeled around a central class named `BankAccount`.

In the design phase, the `BankAccount` class is defined with a private data member called `balance`. This ensures that the account balance cannot be accessed directly from outside the class, thereby maintaining data security through encapsulation. Public methods are then designed to interact with this private data, providing controlled access to perform various operations.

The next step involves implementing the key methods of the system. The `deposit()` method is created to add money to the account. It accepts an amount as input and validates whether the amount is positive before updating the balance. Similarly, the `withdraw()` method is implemented to deduct money from the account. This method includes validation checks to ensure that the withdrawal amount is positive and does not exceed the available balance, thus preventing invalid transactions and overdrafts.

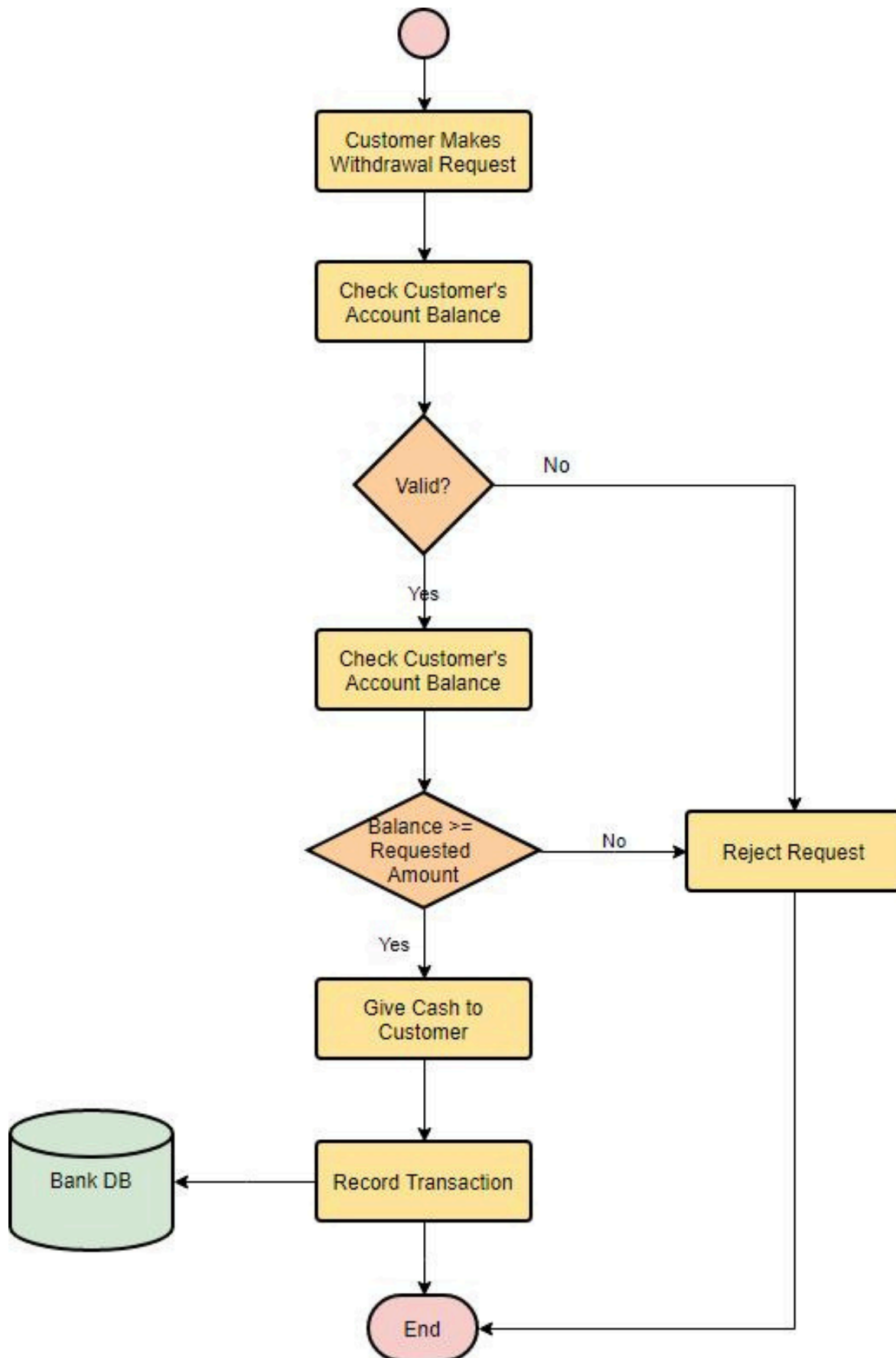
A getter method, `getBalance()`, is also implemented to allow users to view the current account balance without modifying it. This further reinforces the concept of data hiding and controlled access. Additionally, a constructor may be used to initialize the account balance at the time of object creation, providing flexibility in setting initial values.

After implementation, the system is tested by creating objects of the `BankAccount` class in the main method. Various transactions such as deposits and withdrawals are performed to verify the correctness of the system. Both valid and invalid scenarios are tested to ensure that the validation logic works effectively and appropriate messages are displayed for incorrect operations.

Finally, the results are analyzed to confirm that the system behaves as expected. The methodology ensures that the project not only meets its functional requirements but also demonstrates the practical application of Object-Oriented Programming concepts such as encapsulation, modularity, and method interaction.

Overall, this structured approach helps in building a reliable and easy-to-understand system while reinforcing good programming practices and logical problem-solving skills.

Flow Chart:



Implimentation:

The implementation of the Bank Account Management System is carried out using the Java programming language, following the principles of Object-Oriented Programming (OOP). The system is designed in a modular and structured manner, where each functionality is implemented through well-defined classes and methods.

The core of the system is the BankAccount class, which represents a bank account. This class contains a private data member named balance, which is used to store the account balance securely. By declaring this variable as private, direct access from outside the class is restricted, thereby ensuring data protection through encapsulation.

To interact with the account balance, public methods are defined within the class. The deposit() method is implemented to allow users to add money to their account. This method accepts a parameter representing the amount to be deposited. Before updating the balance, the method checks whether the entered amount is greater than zero. If the condition is satisfied, the amount is added to the balance; otherwise, an appropriate error message is displayed, preventing invalid input.

Similarly, the withdraw() method is implemented to handle money withdrawal. This method also takes an amount as input and performs necessary validations. It first checks whether the amount is positive. Then, it verifies whether sufficient balance is available in the account. If both conditions are met, the amount is deducted from the balance. If not, the system displays relevant error messages such as “Invalid amount” or “Insufficient balance,” ensuring safe and accurate transactions.

A getter method named getBalance() is provided to allow users to view the current balance. This method returns the value of the private balance variable without allowing any direct modification, thus maintaining the integrity of the data.

In addition to these methods, a constructor is implemented to initialize the account balance when an object of the BankAccount class is created. This allows the system to start with a predefined balance, making it more flexible and realistic.

The main method serves as the entry point of the program, where objects of the BankAccount class are created. Various operations such as deposit and withdrawal are performed using these objects to simulate real-life banking transactions. The results of these operations, including updated balances and error messages, are displayed on the console.

The implementation is kept simple and console-based, without the use of graphical user interfaces or database connectivity. This ensures that the focus remains on understanding the core programming concepts and logic behind the system.

Overall, the implementation successfully demonstrates the use of encapsulation, method design, and input validation in building a secure and functional Bank Account Management System. It provides a clear example of how theoretical OOP concepts can be applied to develop practical and reliable software solutions.

Future Enhancement:

- The system can be enhanced by adding support for multiple bank accounts, allowing users to create and manage more than one account.
- A database can be integrated to store account details permanently instead of using temporary memory, enabling real-world data management.
- A graphical user interface (GUI) can be developed to make the system more interactive and user-friendly instead of a console-based interface.
- User authentication features such as login ID and password can be added to ensure better security and restricted access.
- Transaction history can be implemented to allow users to view past deposits and withdrawals.
- Additional banking features like interest calculation, fund transfer between accounts, and account statements can be included.
- The system can be upgraded to a web-based or mobile application for remote access and real-time usage.
- Error handling can be improved further using advanced exception handling techniques.
- Notifications or alerts (e.g., low balance warning) can be added to improve user experience.
- Integration with secure technologies such as encryption can be implemented to enhance data protection.
- The project can be expanded into a complete banking management system with modules like loan management, customer profiles, and administrative controls.
- Performance optimization and scalability improvements can be made to handle a large number of users and transactions efficiently.

Overall, these enhancements can transform the basic system into a more advanced, secure, and real-world banking application.

Conclusion:

The Bank Account Management System developed in this project successfully demonstrates the practical application of Object-Oriented Programming principles in solving real-world problems. The system provides a simple yet effective way to manage basic banking operations such as deposit, withdrawal, and balance inquiry while ensuring data security through encapsulation.

One of the key achievements of this project is the implementation of data hiding, where the account balance is protected as a private variable and can only be accessed or modified through controlled public methods. This approach ensures that the integrity of financial data is maintained and prevents unauthorized access. Additionally, proper validation techniques have been applied to handle invalid inputs, such as negative values and insufficient balance, making the system more reliable and robust.

The project also highlights the importance of modular programming and method design. By dividing the system into well-defined classes and methods, the code becomes more organized, readable, and easier to maintain. The use of constructors and object interaction further enhances the flexibility and realism of the system.

Although the current implementation is limited to a console-based application with basic features, it provides a strong foundation for understanding how banking systems operate. It also serves as a stepping stone for developing more advanced applications with additional functionalities such as database integration, graphical interfaces, and enhanced security measures.

In conclusion, this project not only fulfills its intended objectives but also helps in building a clear understanding of core programming concepts and their real-world applications. It has been a valuable learning experience, improving both technical knowledge and problem-solving skills, and preparing the foundation for future software development projects.

Reference:

- Guidance and support provided by my teacher, Mr. Soumik Karmakar.
- Official documentation of Java programming language (Oracle Java Documentation).
- Various online tutorials and educational resources related to Object-Oriented Programming and Java.
- Study materials and notes provided by Rungta International Skills University.
- Books referred:
 - “Java: The Complete Reference” by Herbert Schildt
 - “Object Oriented Programming with Java” by E. Balagurusamy
- Additional help taken from general programming examples and practice problems available in textbooks and online sources.